# End-to-End Compilation and Execution Pipeline for OpenMP Workloads on PULP Architecture

K.V.
Politecnico di Milano
FL and Compilers Project

March 12, 2026

**Abstract**

This report details the development of a complete compiler toolchain and bare-metal OpenMP runtime for deploying multi-core C applications onto the PULP-open architecture via the GVSoC emulator. The pipeline leverages Polygeist for C-to-MLIR translation, LLVM dialect lowering passes, and the PULP LLVM backend for targeted RISC-V assembly generation. Furthermore, a custom bare-metal OpenMP runtime was implemented to support parallel execution across cluster processing elements (PEs). Validation was performed using linear algebra kernels from the PolyBench-ACC suite, demonstrating successful execution of floating-point computations across an 8-core cluster configuration.

## 1 Introduction and Objectives

The primary objective of this project is to establish a unified pipeline capable of taking standard C applications annotated with OpenMP pragmas, compiling them down to RISC-V assembly, and orchestrating their execution on a multi-core PULP (Parallel Ultra Local Processing) platform emulator. The pipeline must handle affine loop transformations, hardware-specific pointer casting, and multi-core synchronization without relying on a full OS.

## 2 Toolchain Architecture

The compilation infrastructure integrates two distinct LLVM-based toolchains to fulfill the requirements of both high-level dialect lifting and hardware-specific backend generation:

- **Polygeist (LLVM 18):** Provides the `cgeist` frontend to parse C/OpenMP code into MLIR affine and SCF dialects. It also supplies `mlir-opt` for lowering MLIR to the LLVM dialect, and `mlir-translate` to convert the LLVM dialect into standard LLVM IR.
- **PULP LLVM (LLVM 15):** Provides the `llc` backend. Because PULP targets use specific hardware extensions (`xpulpv2`, `rv32imfc`), a custom fork of LLVM is required.

## 3 Compilation Pipeline

The automated pipeline script sequentially processes input code through four distinct stages:

1. **C → MLIR:** `cgeist -S -function=* -raise-scf-to-affine` translates the source into MLIR representations.

2. **MLIR Lowering:** `mlir-opt` applies several lowering passes, including `-lower-affine`, `-convert-scf-to-cf`, and `-finalize-memref-to-llvm`, converting high-level loop semantics into the LLVM dialect.

3. **MLIR → LLVM IR:** `mlir-translate -mlir-to-llvmir` yields the intermediate representation.

4. **LLVM IR → RISC-V Assembly:** `llc -march=riscv32 -mattr=+m,+f,+c` processes the IR into assembly targeted for `riscv32-unknown-elf`.

# 4 Bare-Metal Multi-Core OpenMP Runtime

A significant achievement of this project is the custom bare-metal implementation of OpenMP scheduling and synchronization for the PULP-open architecture.

## 4.1 Boot Sequence and Memory Layout

The PULP-open architecture features a Fabric Controller (FC, `mhartid=992`) and a cluster of 9 Processing Elements (PE0-PE8). The boot sequence is structured as follows:

- The FC boots first, configures the cluster boot addresses to the cluster entry point, enables cluster instruction fetching via the `FETCH_EN` register, and then halts, awaiting an End of Computation (EOC) signal via a semihosting trap.

- PE0 operates as the **master thread**. It clears the BSS section, sets up an expanded 16KB stack in the L2 shared memory (0x1C000000 base), and calls `main()`.

- PE1-PE7 operate as **worker threads**, allocating smaller 4KB stacks in the L1 Tightly Coupled Data Memory (TCDM). They spin in a dispatch wait loop upon boot.

## 4.2 Dispatch and Synchronization

When the master thread encounters a `#pragma omp parallel` region, it triggers `__kmpc_fork_call`. PE0 writes the outlined function pointer and parameters to shared L1 TCDM and pushes a wake-up signal to the Event Unit (EU) dispatch FIFO.

Hardware barrier synchronization is managed via the EU global barrier region. Static scheduling is handled via `__kmpc_for_static_init_8u`, which cleanly partitions the iteration space based on the local PE core IDs mapped to global thread IDs.

# 5 Errors Encountered and Resolutions

Building a multi-toolchain bare-metal environment from scratch surfaced several critical integration issues. The errors and their implemented solutions are documented below:

## 5.1 Missing `<cstdint>` in LLVM Codebases

- **Error:** During the build of both Polygeist and PULP LLVM, compilation aborted with errors indicating that types like `uint64_t` and `uintptr_t` were not declared. This occurred because GCC 15 removed implicit inclusions of `<cstdint>` from standard library headers, breaking older LLVM source code.

- **Resolution:** The compiler toolchain was explicitly downgraded. GCC-14 (`g++-14`) was installed to compile Polygeist, and GCC-12 (`g++-12`) was utilized to safely compile the LLVM 15 backend without triggering C++20 standard library conflicts.

### 5.2 GVSoC Target Linker Error

- **Error:** Invoking `cmake` directly in the GVSoC directory resulted in the fatal configuration error: `Cannot specify link libraries for target utils.injector_impl`.
- **Resolution:** Direct CMake invocation was bypassed in favor of the project's native Makefile wrapper. Running `make all TARGETS=pulp-open` correctly satisfied the module dependencies and built the emulator successfully.

### 5.3 Opaque Pointer Incompatibility

- **Error:** Polygeist's frontend (LLVM 18) emits modern "opaque pointers" (`ptr`) in its LLVM IR. When feeding this IR to the initial PULP LLVM 12 backend, the `llc` step crashed with an `expected type` error, as LLVM 12 strictly requires typed pointers (e.g., `i8*`).
- **Resolution:** The PULP LLVM repository was upgraded from the v12 tag to the `15.0.0-snitch-0.5.0` tag. LLVM 15 natively supports opaque pointers, allowing seamless translation from Polygeist's output.

### 5.4 Atomic Operations in Software (Missing 'A' Extension)

- **Error:** The assembly step for the OpenMP runtime failed because `amoadd.w` (atomic add) instructions were generated. The `pulp-open` cluster cores execute the `rv32imfc` ISA, which fundamentally lacks the 'A' (Atomic) hardware extension.
- **Resolution:** The OpenMP synchronization logic and hardware barrier routines were manually rewritten in C/Assembly to remove atomic dependencies, substituting them with lock-free software flags utilizing the L1 TCDM alias space.

### 5.5 64-bit Division Missing on Bare-Metal Target

- **Error:** When linking the final `.elf` binary, the linker complained of undefined references to `__udivdi3` and `__divdi3`. MLIR generated 64-bit loop iterations invoking `__kmpc_for_static_init_8u`, which requires 64-bit division.
- **Resolution:** Because the bare-metal environment lacks a robust standard library, the 64-bit runtime scheduling functions were manually downcast to 32-bit unsigned integers, mapping them directly to the native 32-bit hardware division instructions.

### 5.6 Single Register File Pointer Corruption

- **Error:** During the execution of floating-point GEMM benchmarks, PE0 experienced stack pointer corruption resulting in a silent crash. The GVSoC simulator defaulted to a unified register file where floating-point registers aliased integer registers (e.g., `f2` aliases `x2/sp`).
- **Resolution:** The GVSoC simulator configuration (`pulp_cores.py`) was patched. The `CONFIG_GVSOC_ISS_NO` constraint was removed, granting separate floating-point register files and restoring stack pointer integrity.

## 6 Validation

The complete toolchain and runtime were validated against the OpenMP benchmarks from the PolyBench-ACC suite using the MINI_DATASET configuration.

| Benchmark Kernel | Algorithm Type | Execution Status |
|---|---|---|
| `gemm` | Matrix-Matrix Multiplication | PASS |
| `atax` | Matrix Transpose and Vector Multiplication | PASS |
| `2mm` | Chained Matrix Multiplication | PASS |
| `3mm` | Tri-Chained Matrix Multiplication | PASS |

Table 1: Validation of PolyBench-ACC Kernels on 8-Core PULP-Open Cluster

# 7 Conclusion and Future Work

The implemented toolchain successfully compiles standard OpenMP C code to MLIR, translates it into LLVM IR, and executes parallel, bare-metal RISC-V floating-point operations across a simulated 8-core PULP cluster.

Future optimizations could involve supporting dynamic scheduling algorithms within the OpenMP runtime, enabling DMA (Direct Memory Access) data transfers between L2 memory and L1 TCDM to reduce interconnect latency during kernel execution, and upgrading the toolchain to fully utilize the `xpulpv2` and hardware loop features generated by the backend.